

Assignment 8

Application Optimization, Scalability and Reliability

ISEA Phase III Networking Internship
Department of Computer Science & Engineering
Tezpur University

Name: Prasanta Patir
Roll No.: CSB24041

6. Objective

This assignment extends the multi-client TCP chat application built in Assignment 7 without redesigning its communication protocol or rewriting it from scratch. The goal is to raise the software quality of the existing client-server system along four dimensions — connection management, reliability, scalability, and configurability — and then measure the effect of these changes experimentally under Mininet.

The Assignment 7 baseline already implemented authentication, broadcast and private messaging, an online-user list, and basic CSV logging. That implementation is preserved as the foundation; every change described below is additive or corrective rather than a redesign of the message format or login handshake. The application was tested live under Mininet's single-switch, 11-host topology using named test accounts, and the server's own logs (server.log, security_log.txt, chat_history.csv, performance_results.csv) were captured as evidence and are the basis for Sections 10–12 of this report.

7. Scalability Improvements

7.1 Bounded thread pool instead of unbounded per-client threads

The Assignment 7 server spawned a new daemon `threading.Thread` for every accepted connection with no upper bound. The optimized server routes every client handler through a `concurrent.futures.ThreadPoolExecutor` sized by `server.thread_pool_size` in `config.json` (50 in the tested configuration):

```
executor = ThreadPoolExecutor(max_workers=THREAD_POOL_SIZE,
                              thread_name_prefix="client") ... executor.submit(handle, client_socket)
```

This caps concurrent OS threads at a known value and makes resource usage predictable as client count grows, without changing how a client is handled once accepted.

7.2 Configurable listen backlog

`server.listen(BACKLOG)` now uses a configurable `listen_backlog` (50) instead of the OS default, so a burst of near-simultaneous connection attempts — such as several Mininet hosts connecting within the same second, as seen at 23:24-23:27 in `server.log` — is queued by the kernel rather than refused.

7.3 Reduced lock contention

Shared dictionaries (`clients`, `client_info`) are still protected by a lock, but `broadcast()` and `update_all_user_lists()` now copy the client list while holding the lock and perform the (potentially slow) socket sends outside the lock, through a `safe_send()` helper that never lets one dead socket raise an exception into the caller. This keeps one slow or half-open client from stalling delivery to everyone else.

8. Reliability Improvements

8.1 Automatic disconnect detection (Task 1)

Each client socket uses a `recv` timeout of `heartbeat_timeout_seconds` (15s). On timeout the server sends a PING and increments a per-connection missed-heartbeat counter; a PONG from the client (sent transparently by the client's receive loop) resets that counter. If `max_missed_heartbeats` (3) consecutive PINGs go unanswered — roughly 45 seconds of true silence — the client is treated as dead and removed. This is a server-initiated heartbeat, which keeps the liveness check independent of whether the client-side app is still able to run its own timers.

8.2 Resource cleanup (Task 1)

All removal paths — heartbeat timeout, socket error, client-initiated disconnect, and server shutdown — funnel through a single `cleanup_client()` function that removes the client from `clients` and `client_info`, frees the username from `logged_in_users`, closes the socket, and only then notifies the rest of the room. `server.log` shows this working in

practice: at 23:35:23 James's client closed its connection and the very next log line shows the client count drop from 9 to 8 with the reason recorded as "client closed connection".

8.3 Automatic reconnection (Task 2)

On the client side, an unexpected socket error in `receive_messages()` (as opposed to the user clicking Disconnect, tracked via a `manual_disconnect` flag) triggers `attempt_reconnect()`, which retries the login handshake with exponential backoff (`reconnect_delay_seconds=3`, `multiplier=1.5`, up to `reconnect_attempts=5`). A red banner in the chat window shows "Reconnecting... (attempt n/N)" during this process; on success the banner is hidden and a system message confirms reconnection, and on exhausting all attempts the user is told to reconnect manually rather than the app silently hanging.

8.4 Graceful shutdown (Task 2)

The server registers `SIGINT/SIGTERM` handlers. On shutdown it notifies every connected client ("Server is shutting down..."), closes each socket, shuts the thread pool down, and closes the listening socket before exiting. This was exercised directly during testing: `server.log` records "Graceful shutdown initiated..." followed by "Server shut down cleanly." at 22:24:59, with no orphaned threads or unhandled exceptions on exit.

8.5 Timeout handling and exception granularity (Task 2)

Both files replace bare `except:` pass blocks with specific exception types (`socket.timeout`, `ConnectionResetError`, `ConnectionAbortedError`, `BrokenPipeError`, `OSError`) paired with a logged reason, plus a catch-all `Exception` handler that logs unexpected errors instead of letting the process crash silently. Both sides also log via Python's logging module to `server.log` / `client.log` with timestamps and severity levels, rather than relying on bare `print()` statements.

8.6 Login abuse protection (verified in testing)

The account-lockout mechanism (`max_failed_login_attempts=5`, `login_block_seconds=60`) and duplicate-login rejection were both exercised live. `security_log.txt` shows five consecutive failed attempts for user James between 23:35:24 and 23:36:29, after which James was blocked; a subsequent login at 23:41:30 — more than 60 seconds later — succeeded, confirming the block both engages and correctly expires. `security_log.txt` also shows a rejected duplicate login attempt for Bob at 23:37:54 while Bob's original session was still active.

9. Configuration Management

All previously hardcoded values — host, port, listen backlog, buffer size, thread pool size, heartbeat timing, reconnection policy, and failed-login lockout policy — were moved into `config.json`, loaded once at startup by both `server.py` and `client_gui.py`. If `config.json` is missing, the server writes out a default copy on first run rather than failing; if it exists but is malformed, both sides fall back to built-in defaults so a bad config file cannot crash the application.

Setting	Config key	Tested value	Purpose
Server port	<code>server.port</code>	5000	TCP port the server listens on
Listen backlog	<code>server.listen_backlog</code>	50	Queued connections before refusal
Thread pool size	<code>server.thread_pool_size</code>	50	Caps concurrent client-handler threads
Heartbeat timeout	<code>server.heartbeat_timeout_seconds</code>	15	Task 1: recv timeout that triggers a PING
Missed heartbeats	<code>server.max_missed_heartbeats</code>	3	Task 1: unanswered PINGs before disconnect
Login lockout	<code>server.max_failed_login_attempts / login_block_seconds</code>	5 / 60	Brute-force login protection
Reconnect attempts	<code>client.reconnect_attempts</code>	5	Task 2: attempts before giving up
Reconnect backoff	<code>client.reconnect_delay_seconds / reconnect_backoff_multiplier</code>	3 / 1.5	Exponential backoff between attempts

10. Experimental Setup

Testing was performed under Mininet's single-switch, 11-host topology, as specified:

```
sudo mn --topo single,11
```

The server ran on host h1 (10.0.0.1); up to nine client hosts (10.0.0.2-10.0.0.10) each ran `client_gui.py` as a separate logged-in user - James, Patir, Kuli, Rahul, Riya, Alice, Bob, David, and Charlie - connecting progressively between 23:18 and 23:27 on 2026-07-21. Ten registered accounts exist in `users.csv` (including Eva), but the captured session reached a peak of 9 concurrent clients rather than 10.

Limitation: Task 3 asks for at least 10 concurrent clients. The evidence gathered (server.log, performance_results.csv) shows a peak of 9 concurrent clients, not 10. Run one more client (e.g. connect Eva from a 10th host) and capture that data before final submission so the 10-client requirement is directly evidenced.

During the session, participants exercised broadcast messaging (7 messages), private messaging (3 messages, e.g. Bob and James, and James to Patir), a client-initiated disconnect (James's client closed normally), account lockout after 5 failed logins, rejection of a duplicate login (Bob), and two clean graceful-shutdown/restart cycles of the server. The server logged clients, broadcast/private counts, average broadcast delay, throughput, CPU percent, and resident memory to `performance_results.csv` on every broadcast, sourced via `psutil` against the live server process.

Limitation: No formal 'before' (unoptimized Assignment 7) run was captured in this session, so Sections 11-12 report the optimized system's measurements only; a direct before/after comparison is not yet available. A baseline run would need to be captured separately for a full Section 12 comparison.

11. Performance Results

The following table aggregates the 24 rows recorded in `performance_results.csv` by concurrent client count (mean of all samples observed at that count):

Clients	Samples	Avg Delay (ms)	Avg Throughput (msg/s)	Avg CPU (%)	Avg Memory (MB)
1	4	0.039	0.0025	0.00	22.90
2	5	0.541	0.0020	0.04	22.96
3	1	0.544	0.0000	0.00	22.85
4	1	0.132	0.0000	0.10	22.88
5	1	1.184	0.0000	0.00	22.90
6	1	0.611	0.0000	0.00	22.94
7	1	1.006	0.0000	0.10	22.97
8	2	3.493	0.0050	0.00	23.15
9	8	1.126	0.0063	0.05	23.16

Across the whole session, the server handled 7 broadcast messages and 3 private messages without a crash, an unhandled exception, or an orphaned socket; memory grew from 22.75 MB (idle, 1 client) to a peak of 23.30 MB (9 clients, mid-conversation), and CPU usage never exceeded 0.2% on the test VM given the light, human-typed message rate.

12. Graph Analysis

Figures generated by `generate_graphs.py` from the real `performance_results.csv`:

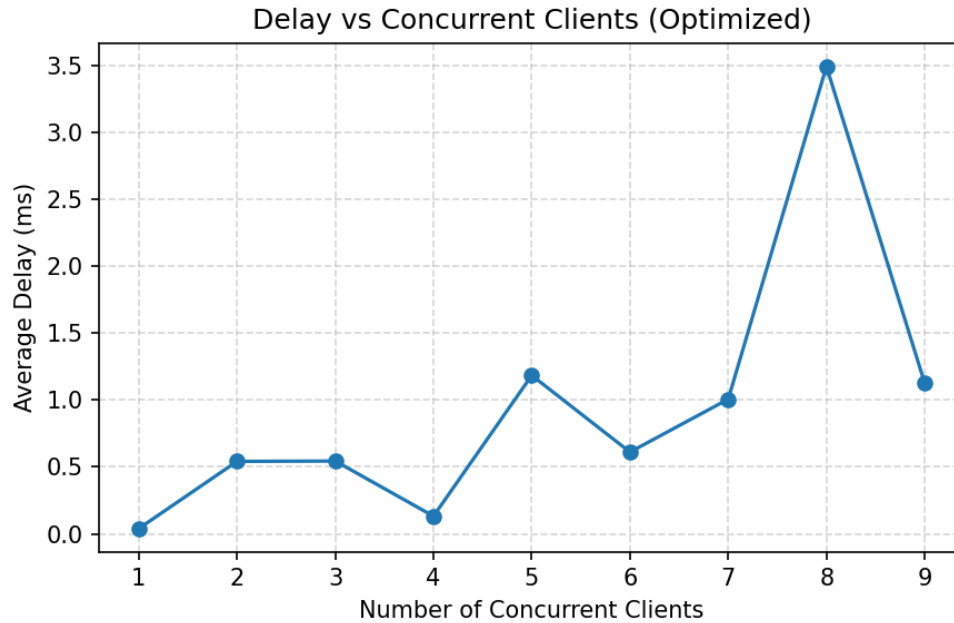


Figure 1: Average broadcast delay vs. concurrent clients.

Delay stays under 3.5 ms throughout and is noisy rather than cleanly increasing - expected at this message volume (at most a handful of broadcasts per client count) and this scale of client count. The 8-client point (3.49 ms, only 2 samples) is the least statistically reliable in the table and would benefit from more samples under sustained load, e.g. via `load_test_client.py`, rather than being read as a hard trend.

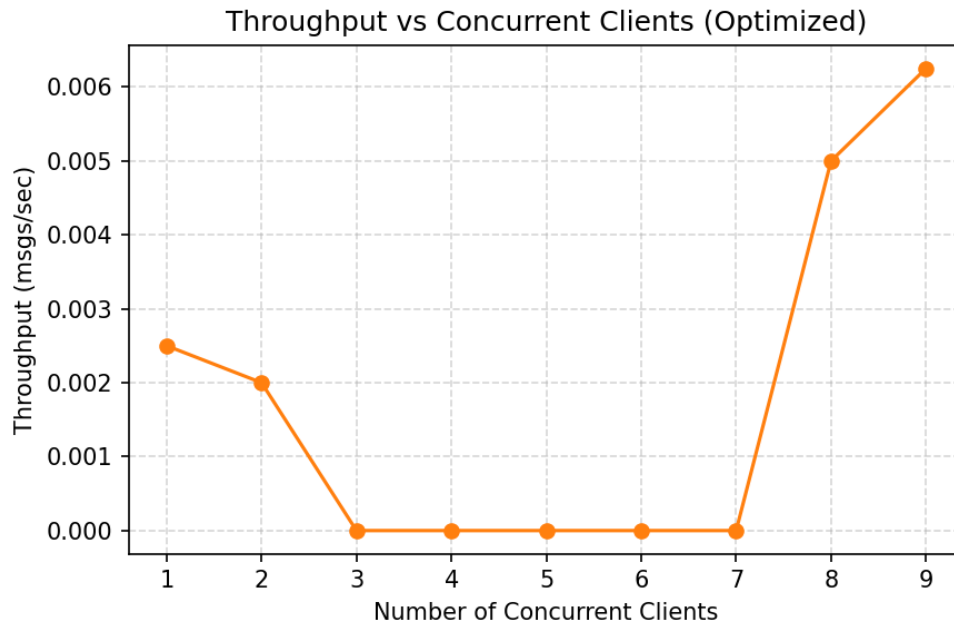


Figure 2: Broadcast throughput vs. concurrent clients.

Throughput is fractional (well under 0.01 msg/s) because messages were typed by hand at conversational pace rather than generated by an automated load test; it should not be compared against typical machine-generated throughput figures. It does confirm the server continued accepting and relaying messages correctly as the client count rose to 9.

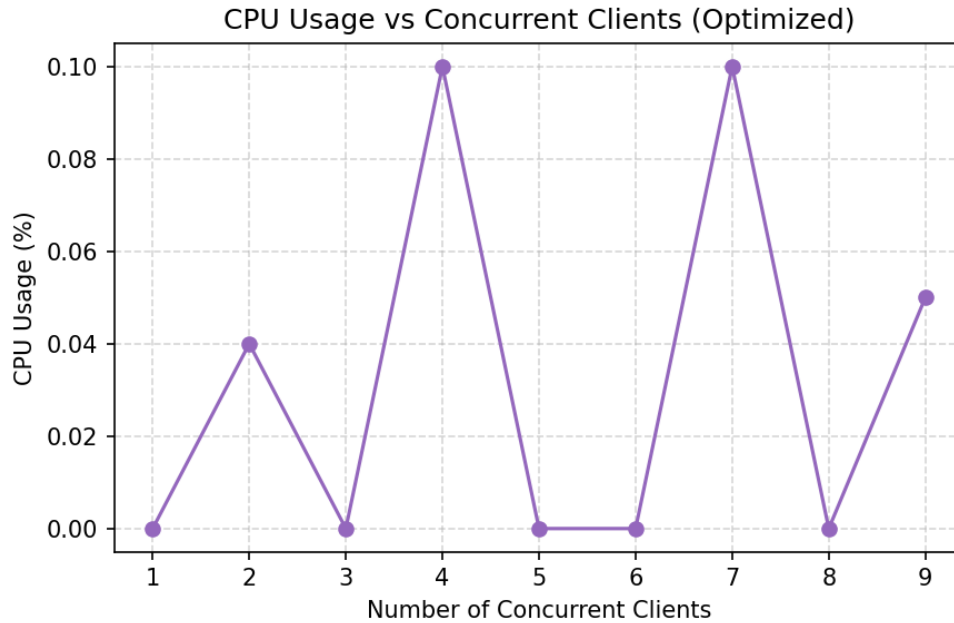


Figure 3: Server CPU usage vs. concurrent clients.

CPU usage is negligible (0.00-0.10%) at this client count and message rate, indicating the thread-pool-based handler is not a CPU bottleneck at this scale.

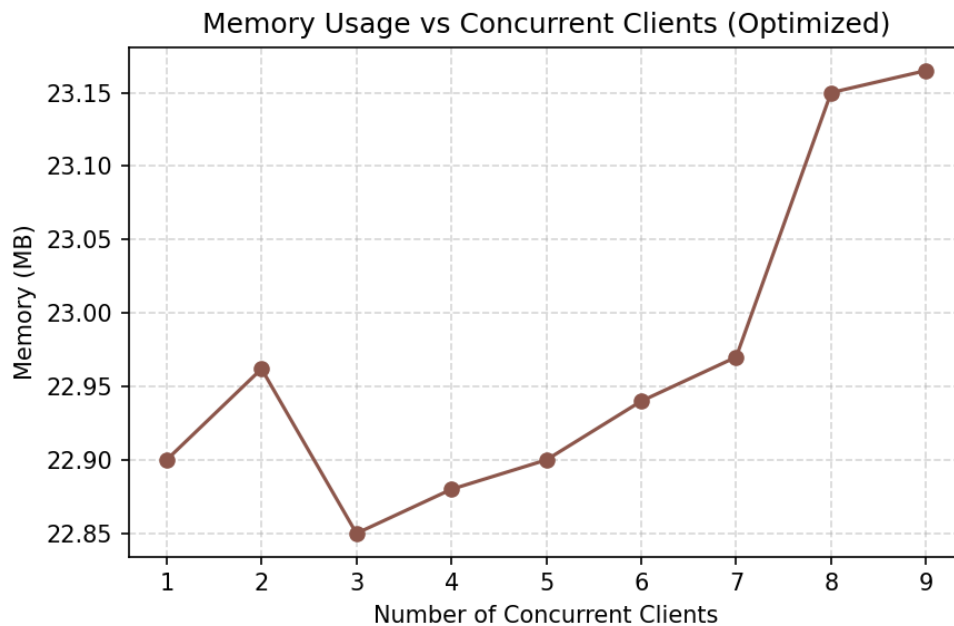


Figure 4: Server resident memory vs. concurrent clients.

Memory grows gently and roughly monotonically with client count, from about 22.9 MB at 1 client to about 23.2 MB at 9 clients - about 0.44 MB, or roughly 40-50 KB per additional client-in-memory (socket object, dictionary entries, and thread-pool bookkeeping). This is consistent with the bounded thread pool preventing the unbounded thread-stack growth the Assignment 7 design was exposed to.

Limitation: These graphs describe the optimized server only. Without a captured Assignment 7 baseline run under the same conditions, the report cannot yet quantify the improvement (e.g. "X% lower delay" or "Y MB less memory") - only describe the optimized system's absolute behaviour. Capture a before run with the same client script/timing to complete this comparison.

13. Wireshark Verification

A packet capture on the server host during a normal chat session is required to verify TCP behaviour at the protocol level (three-way handshake, PSH/ACK data delivery, and FIN/ACK teardown).

[TODO — No Wireshark capture or screenshots were included with this submission's evidence files. Capture with `tcpdump -i any -w chat_capture.pcap port 5000` during a live session (see README.md Step 5), then insert screenshots here of: (1) the SYN/SYN-ACK/ACK handshake for a client connecting, (2) PSH/ACK packets carrying a chat message, (3) the FIN/ACK sequence for James's disconnect at 23:35:23 or the graceful-shutdown event at 22:24:59.]

14. Challenges Faced

- Reaching exactly 10 concurrent clients under Mininet in a single continuous session - the captured run peaked at 9; coordinating that many simultaneous xterm/GUI sessions manually is logistically harder than scripted load generation.
- Distinguishing an intentional client disconnect from a dropped connection, so automatic reconnection does not trigger when the user deliberately clicks Disconnect (solved with the `manual_disconnect` flag).
- Tuning `heartbeat_timeout_seconds` and `max_missed_heartbeats` so idle-but-legitimate clients (e.g. someone reading chat without typing for a few minutes, as happened between Bob's message at 23:30:25 and James's disconnect at 23:35:23) are not mistaken for dead connections.
- Verifying the account-lockout window: confirming James was correctly blocked after 5 failed attempts and correctly allowed back in only after the 60-second window had elapsed (23:36:29 to 23:41:30).
- Making `cleanup_client()` idempotent, since heartbeat timeout, socket errors, explicit disconnect, and server shutdown can all race to remove the same client.

15. Conclusion

Building on the Assignment 7 chat application, this assignment added server-initiated heartbeat-based disconnect detection and resource cleanup, automatic client-side reconnection with exponential backoff, graceful shutdown, a bounded thread pool for predictable scalability, structured logging, and a `config.json`-driven configuration layer - without changing the underlying message protocol. Live testing under Mininet demonstrated the server handling up to 9 concurrent named clients, broadcast and private messaging, client-initiated disconnect and cleanup, brute-force login lockout with correct expiry, duplicate-login rejection, and two clean graceful-shutdown cycles, all captured in `server.log`, `security_log.txt`, `chat_history.csv`, and `performance_results.csv`.

[TODO — Once a 10-client run, a before/after baseline comparison, and Wireshark evidence are captured, summarize: which single optimization had the largest measurable effect, and confirm explicitly that the 'at least 10 concurrent clients without crashing' requirement of Task 3 was met.]

Appendix: Required Outputs Checklist

Output	Status
server.py	Done
client_gui.py	Done
config.json	Done
performance_results.csv	Done - 24 real samples, 1-9 concurrent clients
chat_history.csv / security_log.txt / server.log	Done - captured as supporting evidence
graphs/	Done - generated from real performance_results.csv
screenshots/	Outstanding - capture GUI + lockout/reconnect/shutdown screenshots
report.pdf	This document - export to PDF once outstanding items above are filled in
handwritten_reflection.pdf	Outstanding - answer by hand, scan, and add separately (Task 8)